

Momentum MCP

Hiring Manager Brief

Your AI remembers where you left off.

In one line: I built a local-first continuity layer so AI assistants can retrieve open loops, project context, people, and decision rationale instead of relying only on chat history.

Why I built it

I did not start with "I want to build an MCP server." I started with a problem I kept seeing in my own work: creating tasks was easy, but the context around them kept getting lost. A follow-up might live in one note, a decision in another conversation, and the reason behind a project choice somewhere else. When I came back later, I had to rebuild the story before I could move forward.

The idea: what if an AI assistant could remember where I left off without needing my whole workflow uploaded to another cloud service?

The problem it solves

Momentum is not meant to replace a task manager. It preserves the context around work: what I am working on, what I am waiting on, who is involved, what decisions were made, and why. The goal is to reduce the time spent reconstructing context.

Main use cases

- Daily brief - open loops, priorities, overdue items, and waiting items.
- Waiting on someone - track an external dependency separately from a normal task.
- Project handoff to myself - ask, "Where did I leave this?" and recover recent context and decisions.
- Decision memory - save both the decision and the reason behind it.
- Search - find related notes, tasks, people, and decisions.

How someone can use it

Local CLI: use Momentum without an AI client - for example `momentum brief`, `momentum waiting`, or `momentum context --project "Momentum MCP"`.

MCP client: an MCP-compatible AI can call the Momentum tools directly. Momentum supplies structured state; the AI handles reasoning.

Tech stack

TypeScript for the server/CLI, **Node.js 24** for runtime, **MCP** for the AI tool interface, **SQLite** for local persistence, **Zod** for input validation, and **GitHub Actions** for build/tests/privacy checks.

Architecture in plain English: **AI client or local CLI -> Momentum service -> local SQLite database.**

Why MCP instead of just another productivity app?

MCP lets me separate **reasoning** from **state**. Momentum owns the durable state. The AI client owns the reasoning. That means the product is not tied to one model or one chatbot.

- Portability - the same context can work underneath different MCP-compatible AI clients.
- Tool boundaries - I can expose narrow actions such as "get waiting items" or "record decision" instead of unrestricted database access.
- Model independence - the reasoning layer can change without losing the user's state.

Why not just use a task manager?

Because the important unit is not always a task. Sometimes I need to preserve an external dependency, a project state, a person connected to a follow-up, or the reason behind a decision. Momentum treats those as first-class context instead of forcing everything into a checklist.

Privacy choices

- Real data stays local by default.
- The private database lives outside the Git repository.
- The core project has no telemetry and does not require an LLM API key.
- Public tests and demos use fictional data only.
- A privacy check scans for common secrets and private database/key files before publishing.

Why this matters to me as a PMM

The most useful part of building Momentum was seeing how technical choices become product marketing choices. Context, permissions, privacy, and tool boundaries are not just engineering details - they affect positioning, trust, adoption, and the value a user actually experiences.

Your AI remembers where you left off.

What I personally owned

- Defined the user problem and product hypothesis.
- Reduced the data model to projects, people, context, open loops, and decisions.
- Chose MCP and a local-first architecture.
- Built the server, CLI, persistence layer, tests, documentation, and privacy guardrails.
- Created fictional demo data so the project could be shared publicly without exposing personal information.

60-second interview answer

I built Momentum MCP after noticing that my biggest productivity cost was not creating tasks, but rebuilding context when I returned to work. I wanted a local-first layer that could store open loops, project context, people, and decision rationale, and make that state available to an AI through MCP. I built it in TypeScript on Node with SQLite, added a local CLI so it is useful even without an AI client, and designed privacy into the architecture by keeping real data outside the repo. What made the project especially useful for me as a PMM was seeing how technical decisions around context, permissions, and privacy directly shape positioning, trust, and adoption.

Next: better ranking, deduplication, optional calendar/email connectors with explicit permissions, encrypted backup/export, and a lightweight visual interface.

What I want a hiring manager to take away: I notice a real workflow problem, reduce it to a clear product idea, make technical tradeoffs, build a working version, think about privacy and trust, and translate the product into a story users and buyers can understand.

Public project: github.com/pratham-sharma-10/Projects/tree/main/momentum-mcp